Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра электронных вычислительных машин

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 1
Basic REST service

по дисциплине «Программирование на языках высокого уровня»

Выполнил ст. гр. 450503 А.П. Красько

Проверил асс. каф. ЭВМ И.Г. Скиба

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

1. Создать Spring Boot приложение.
2. Реализовать REST API для одной ключевой сущности своей предметной области (domain).
3. Реализовать:
- GET endpoint с @RequestParam
- GET endpoint с @PathVariable
4. Реализовать слои: Controller → Service → Repository.
5. Реализовать DTO и mapper между Entity и API-ответом.
6. Настроить Checkstyle и привести код к стилю.

2 СТРУКТУРА ПРОЕКТА

```
src
└── main
    ├── java
    │   └── com
    │       └── kruosant
    │           └── bookwalker
    │               ├── BookwalkerApplication.java
    │               ├── controllers
    │               │   └── BookController.java
    │               ├── domains
    │               │   └── Book.java
    │               ├── dtos
    │               │   ├── BookCreateDto.java
    │               │   └── BookFullDto.java
    │               ├── exceptions
    │               │   ├── ErrorResponse.java
    │               │   ├── GlobalExceptionHandler.java
    │               │   └── ResourceNotFoundException.java
    │               ├── mappers
    │               │   └── BookMapper.java
    │               ├── repositories
    │               │   └── BookRepository.java
    │               └── services
    │                   └── BookService.java
    └── resources
        ├── application.properties
        ├── static
        └── templates
```

3 ЛИСТИНГ КОДА

Файл BookwalkerApplication.java:

```
package com.kruosant.bookwalker;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration;


@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
public class BookwalkerApplication {
  static void main(String[] args) {
    SpringApplication.run(BookwalkerApplication.class, args);
  }

}
```

Файл ErrorResponse.java:

```
package com.kruosant.bookwalker.exceptions;


import java.time.LocalDateTime;

public record ErrorResponse(int status, String message, LocalDateTime
timestamp) {

}
```

Файл ResourceNotFoundException.java:

```
package com.kruosant.bookwalker.exceptions;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
  public ResourceNotFoundException(String message) {
    super(message);
  }
}
```

Файл GlobalExceptionHandler.java:

```
package com.kruosant.bookwalker.exceptions;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

import java.time.LocalDateTime;

@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(ResourceNotFoundException.class)
  public ResponseEntity<ErrorResponse>
handleResourceNotFound(ResourceNotFoundException ex) {
    ErrorResponse error = new ErrorResponse(
        HttpStatus.NOT_FOUND.value(),
        ex.getMessage(),
        LocalDateTime.now()
    );
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
  }


}
```

Файл BookCreateDto.java:

```
package com.kruosant.bookwalker.dtos;

import lombok.*;

import java.util.Date;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class BookCreateDto {

  private String name;
  private String author;
  private long pageCount;
  private Date publishDate;
  private String publisher;

}
```

Файл BookFullDto.java:

```
package com.kruosant.bookwalker.dtos;

import lombok.*;
```

```
import java.util.Date;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class BookFullDto {
  private long id;
  private String name;
  private String author;
  private long pageCount;
  private Date publishDate;
  private String publisher;
}
```

Файл BookController.java:

```
package com.kruosant.bookwalker.controllers;

import org.springframework.web.bind.annotation.*;
import com.kruosant.bookwalker.domains.Book;
import com.kruosant.bookwalker.dtos.BookCreateDto;
import com.kruosant.bookwalker.dtos.BookFullDto;
import com.kruosant.bookwalker.mappers.BookMapper;
import com.kruosant.bookwalker.services.BookService;

import java.util.List;

@RestController
public final class BookController {
  private final BookService service;
  private final BookMapper mapper;

  public BookController(BookService service, BookMapper mapper) {
    this.service = service;
    this.mapper = mapper;
  }

  @GetMapping("/book/{id}")
  public BookFullDto getById(@PathVariable Long id) {
    return service.getById(id);
  }

  @GetMapping("/search")
  public List<BookFullDto> getByName(@RequestParam String name) {
    return service.getAllByName(name);
  }

  @GetMapping("/books")
  public List<BookFullDto> getAll() {
    return service.getAll();
```

```
  }

  @PostMapping("/book")
  public BookFullDto addBook(@RequestBody BookCreateDto bookd) {
    final Book book = mapper.toBook(bookd);
    return service.create(book);
  }

}
```

Файл BookService.java:

```
package com.kruosant.bookwalker.services;


import com.kruosant.bookwalker.domains.Book;
import com.kruosant.bookwalker.dtos.BookFullDto;
import com.kruosant.bookwalker.exceptions.ResourceNotFoundException;
import com.kruosant.bookwalker.mappers.BookMapper;
import com.kruosant.bookwalker.repositories.BookRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;


@Service
public class BookService {
  private final BookRepository repo;
  private final BookMapper mapper;

  public BookService(BookRepository repo, BookMapper mapper) {
    this.repo = repo;
    this.mapper = mapper;
  }

  public List<BookFullDto> getAllByName(String name) {
    List<Book> books = repo.findByName(name);
    if (books.isEmpty()) {
      throw new ResourceNotFoundException("Not found");
    }

    return books.stream().map(mapper::toBookDto).toList();
  }

  public BookFullDto getById(long id) {
    Optional<Book> optionalBook = repo.findFirstById(id);
    if (optionalBook.isEmpty()) {
      throw new ResourceNotFoundException("Not found");
    }
    return mapper.toBookDto(optionalBook.get());
  }
```

```
  public BookFullDto create(Book book) {
    return mapper.toBookDto(repo.save(book));
  }

  public List<BookFullDto> getAll() {
    return mapper.toBookDto(repo.getAll());
  }
}
```

Файл Book.java:

```
package com.kruosant.bookwalker.domains;

import lombok.*;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import java.util.Date;


@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Book {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private long id;
  private String name;
  private String author;
  private long pageCount;
  private Date publishDate;
  private String publisher;

}
```

Файл BookMapper.java:

```
package com.kruosant.bookwalker.mappers;

import com.kruosant.bookwalker.domains.Book;
import com.kruosant.bookwalker.dtos.BookCreateDto;
import com.kruosant.bookwalker.dtos.BookFullDto;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;
```

```java
@Service
public class BookMapper {
  public Book toBook(BookFullDto bdto) {
    if (bdto == null) {
      return null;
    }

    return Book.builder()
        .id(bdto.getId())
        .name(bdto.getName())
        .author(bdto.getAuthor())
        .pageCount(bdto.getPageCount())
        .publishDate(bdto.getPublishDate())
        .publisher(bdto.getPublisher())
        .build();
  }

  public Book toBook(BookCreateDto bdto) {
    if (bdto == null) {
      return null;
    }

    return Book.builder()
        .id(0)
        .name(bdto.getName())
        .author(bdto.getAuthor())
        .pageCount(bdto.getPageCount())
        .publishDate(bdto.getPublishDate())
        .publisher(bdto.getPublisher())
        .build();
  }

  public BookFullDto toBookDto(Book book) {
    if (book == null) {
      return null;
    }

    return BookFullDto.builder()
        .id(book.getId())
        .name(book.getName())
        .author(book.getAuthor())
        .pageCount(book.getPageCount())
        .publishDate(book.getPublishDate())
        .publisher(book.getPublisher())
        .build();
  }

  public Optional<BookFullDto> toBookDto(Optional<Book> optBook) {
    return optBook.map(this::toBookDto);
  }

  public List<BookFullDto> toBookDto(List<Book> books) {
    return books.stream().map(this::toBookDto).toList();
  }
}
```

Файл BookRepository.java:

```
package com.kruosant.bookwalker.repositories;

import com.kruosant.bookwalker.domains.Book;
import org.springframework.stereotype.Repository;

import java.sql.Date;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;


@Repository
public class BookRepository {
  List<Book> books = new ArrayList<>();

  public BookRepository() {
    books.add(Book.builder()
        .id(1)
        .name("Real time rendering")
        .author("Erick Heine")
        .pageCount(1000)
        .publishDate(Date.valueOf(LocalDate.parse("2018-06-25")))
        .publisher("A K Peters/CRC Press")
        .build());

    books.add(Book.builder()
        .id(2)
        .name("Clean Code: A Handbook of Agile Software Craftsmanship")
        .author("Robert C. Martin")
        .pageCount(464)
        .publishDate(Date.valueOf(LocalDate.parse("2008-08-01")))
        .publisher("Prentice Hall")
        .build());

    books.add(Book.builder()
        .id(3)
        .name("Design Patterns: Elements of Reusable Object-Oriented
Software")
        .author("Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides")
        .pageCount(395)
        .publishDate(Date.valueOf(LocalDate.parse("1994-10-21")))
        .publisher("Addison-Wesley Professional 1")
        .build());

    books.add(Book.builder()
        .id(4)
        .name("The Pragmatic Programmer: Your Journey to Mastery")
        .author("David Thomas, Andrew Hunt")
        .pageCount(352)
```

```
        .publishDate(Date.valueOf(LocalDate.parse("2019-07-30")))
        .publisher("Addison-Wesley Professional pb")
        .build());

    books.add(Book.builder()
        .id(5)
        .name("Introduction to Algorithms")
        .author("Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest,
Clifford Stein")
        .pageCount(1312)
        .publishDate(Date.valueOf(LocalDate.parse("2009-07-31")))
        .publisher("MIT Press")
        .build());

    books.add(Book.builder()
        .id(6)
        .name("The Mythical Man-Month: Essays on Software Engineering")
        .author("Frederick P. Brooks Jr.")
        .pageCount(336)
        .publishDate(Date.valueOf(LocalDate.parse("1995-08-12")))
        .publisher("Addison-Wesley Professional")
        .build());
  }

  public Optional<Book> findFirstById(long id) {
    return books.stream().filter((Book b) -> b.getId() == id).findFirst();
  }

  public List<Book> findByName(String name) {
    return books.stream().filter(b -> b.getName().equals(name)).toList();

  }

  public Book save(Book book) {
    if (book.getId() == 0) {
      book.setId((long) books.size() + 1);
    }
    books.add(book);
    return book;
  }

  public List<Book> getAll() {
    return books;
  }
}
```

4 РЕЗУЛЬТАТ ПРОХОЖДЕНИЯ CHECKSTYLE

```
kruosant@nitro-5  ~/java/bookwalker  main +  mvn checkstyle:check
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.kruosant:bookwalker >-----------------------
[INFO] Building bookwalker 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- checkstyle:3.6.0:check (default-cli) @ bookwalker ---
[INFO] You have 0 Checkstyle violations.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.070 s
[INFO] Finished at: 2026-02-25T12:29:38+03:00
[INFO] ------------------------------------------------------------------------
```

Рисунок 4.1 – Результат прохождения checkstyle

5 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

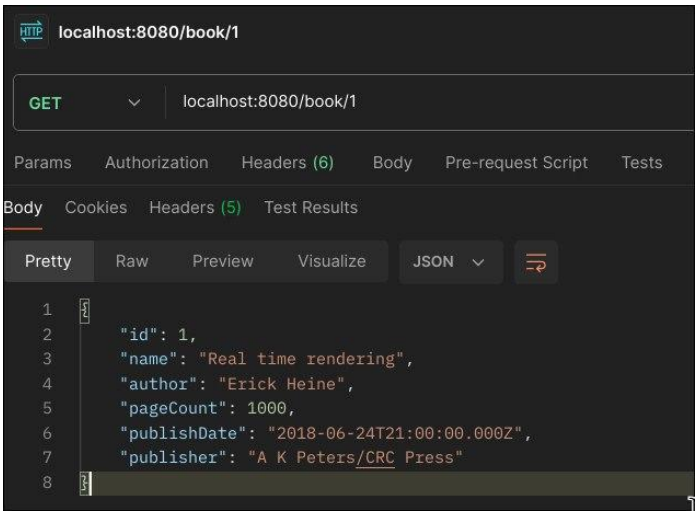

Рисунок 5.1 – Результат работы GET endpoint с @PathVariable

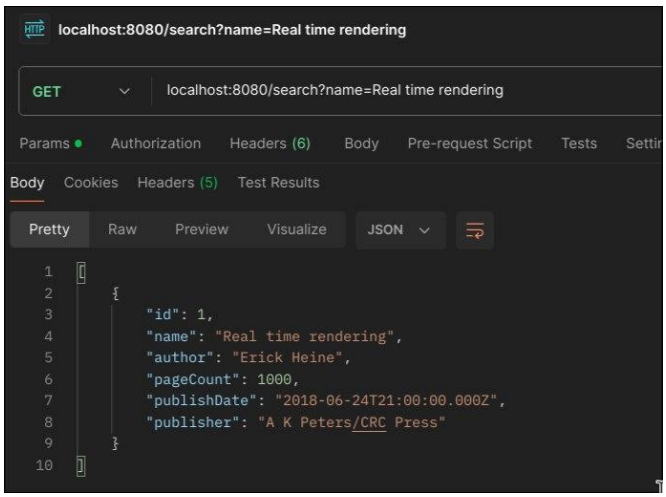

Рисунок 5.2 – Результат работы GET endpoint с @RequestParam

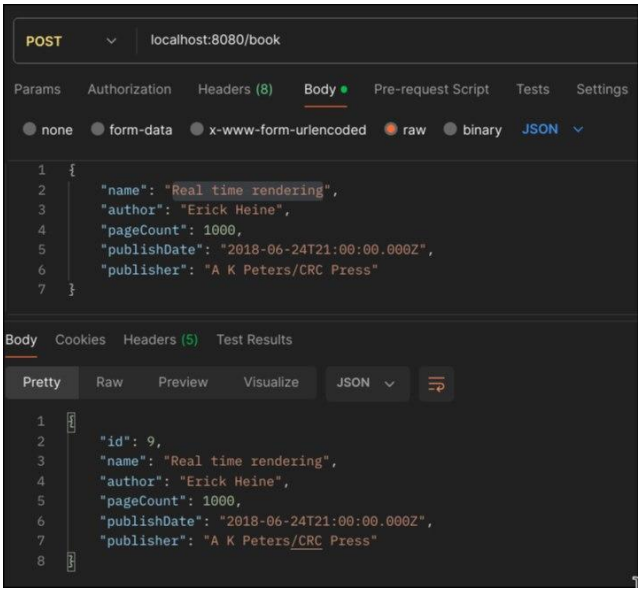

Рисунок 5.3 – Результат работы POST endpoint

6 ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было разработано Spring Boot приложение, реализующее REST API для работы с сущностью «Book». В приложении созданы два основных эндпоинта: с использованием @RequestParam для поиска и с @PathVariable для получения конкретного ресурса. Была успешно применена многослойная архитектура (Controller — Service — Repository), что обеспечило разделение ответственности и упростило поддержку кода. Для передачи данных между слоями использовались DTO (Data Transfer Object), а преобразование между сущностью и DTO выполнялось с помощью специального mapper-класса, что позволило изолировать внутреннюю модель данных от внешнего API. Заключительным этапом стала настройка Checkstyle и приведение кода в соответствие с установленными стандартами кодирования, что повысило читаемость и качество исходного кода. Цели работы достигнуты, основные принципы разработки на Spring Boot закреплены на практике.